

PROPOSTA DE MELHORIAS E ROADMAP

Fabulosa Modas - Evolução para E-commerce Completo

Gerado em: 10/08/2026, 15:35:29

1. RESUMO EXECUTIVO DAS MELHORIAS

Este documento propõe uma evolução arquitetural e funcional para transformar a landing page estática em um e-commerce escalável, mantendo a identidade visual e UX atuais. As melhorias são organizadas em 5 pilares: Arquitetura, Funcionalidades Core, Performance/SEO, Qualidade/DevOps e Acessibilidade.

2. PILAR 1: ARQUITETURA E REFACTORING

2.1 Migração para TypeScript

- Renomear .jsx !' .tsx, adicionar tsconfig.json
- Tipar props de todos os componentes (Product, Collection, CartItem)
- Criar types/shared para contratos de API
- Habilitar strict mode no TypeScript

2.2 Separação de Responsabilidades

- Extrair ProductCard para src/components/ui/ProductCard.tsx
- Criar ProductModal.tsx separado
- Mover collections para src/data/collections.ts (ou CMS/API)
- Criar hooks customizados: useScrollToSection, useProductModal, useCollections
- Implementar Context API para CartState + WhatsApp config

2.3 Estrutura de Pastas Proposta

```
src/
% % % components/
%   % % % ui/                                # Componentes base reutilizáveis
%   %   % % % Button.tsx
%   %   % % % Modal.tsx
%   %   % % % ProductCard.tsx
%   %   % % % LoadingSpinner.tsx
%   % % % layout/
%   %   % % % Navbar.tsx
%   %   % % % Footer.tsx
%   % % % sections/
%       % % % Hero.tsx
%       % % % About.tsx
%       % % % ProductCatalog.tsx
% % % hooks/
%   % % % useScrollToSection.ts
%   % % % useProductModal.ts
%   % % % useCart.ts
% % % context/
%   % % % CartContext.tsx
%   % % % ConfigContext.tsx
% % % data/
```

```
%    % % % collections.ts          # Dados tipados (ou fetch de API)
%    % % % products.ts
% % % types/
%    % % % product.ts
%    % % % cart.ts
%    % % % api.ts
% % % utils/
%    % % % formatters.ts
%    % % % validators.ts
% % % styles/
% % % % % globals.css
```

3. PILAR 2: FUNCIONALIDADES CORE DE E-COMMERCE

3.1 Carrinho de Compras Completo

- Context API + localStorage para persistência
- Operações: add, remove, updateQuantity, clear
- Badge no ícone da navbar com contagem
- Drawer lateral (slide-over) ou página /carrinho
- Cálculo de subtotal, frete, total
- Validação de estoque (mock ou API)

3.2 Checkout via WhatsApp Aprimorado

- Mensagem formatada com lista de itens, quantidades, total
- Configuração de número via variável de ambiente (VITE_WHATSAPP_NUMBER)
- Opção de "Finalizar compra no site" (futura integração gateway)
- Geração de pedido com ID único para rastreamento

3.3 Catálogo Inteligente

- Busca em tempo real (debounced) por nome, marca, categoria
- Filtros: gênero, faixa de preço, marca, tamanho, cor
- Ordenação: preço (menor/maior), novidades, mais vendidos
- Paginação (12 por página) ou Infinite Scroll com IntersectionObserver
- Estados vazios: "Nenhum produto encontrado", "Erro ao carregar"

3.4 Páginas de Produto (PDP)

- Rota dinâmica /produto/:id com React Router
- Galeria de imagens (thumbnails + zoom)
- Seletor de tamanho/cor com estoque por variante
- Descrição rica, especificações, tabela de medidas
- Produtos relacionados (cross-sell)
- Botão "Compartilhar" (Web Share API + fallback)

3.5 Autenticação e Perfil (Fase 2)

- Login social (Google, Facebook) + email/senha
- Histórico de pedidos
- Endereços salvos
- Wishlist / Favoritos sincronizados

4. PILAR 3: PERFORMANCE, SEO E CORE WEB VITALS

4.1 Otimizações de Build

- Code-splitting por rota: lazy(() => import("./pages/ProductCatalog"))
- Tree-shaking: remover lucide-react icons não usados (importar individualmente)
- Compressão Brotli/Gzip no servidor
- Bundle analyzer: npm run build -- --analyze

4.2 Imagens e Assets

- Migração para <Image> do React (ou componente customizado com srcset)
- Geração automática de WebP/AVIF em múltiplas larguras (sharp/Vite plugin)
- Lazy loading nativo (loading="lazy") + placeholder blur (LQIP)
- Preload apenas hero images; demais lazy
- CDN para assets estáticos (Cloudflare R2, AWS S3 + CloudFront)

4.3 SEO Técnico

- Meta tags dinâmicas por rota (react-helmet-async)
- Open Graph + Twitter Cards para compartilhamento
- JSON-LD Product / Breadcrumb / Organization schema
- Sitemap.xml gerado no build (vite-plugin-sitemap)
- robots.txt
- Canonical URLs
- Migração futura para Next.js/Remix para SSR/SSG real

4.4 Core Web Vitals Targets

	Atual	Meta	Como Melhorar
LCP	~3.5s	< 2.5s	Preload hero, otimizar fontes, CDN
INP	~200ms	< 200ms	Code-splitting, reduzir main thread
CLS	~0.15	< 0.1	Reservar espaço para imagens, fonts
FCP	~1.8s	< 1.8s	Critical CSS inline, font-display: swap
TTFB	~600ms	< 600ms	Edge caching, otimizar servidor

5. PILAR 4: QUALIDADE, TESTES E DEVOPS

5.1 Testes

- Unitários: Vitest + React Testing Library (hooks, utils, components)
- Integração: fluxo carrinho !' checkout WhatsApp
- E2E: Playwright (cenários: navegação, busca, adicionar ao carrinho, mobile)
- Visual Regression: Chromatic ou Playwright screenshot testing
- Coverage target: >80% em lógica de negócio (hooks, context, utils)

5.2 Linting, Formatação e Git Hooks

- ESLint + Prettier configurados (airbnb-typescript ou similar)
- Husky + lint-staged: pre-commit (lint + format + typecheck)
- Commitlint: conventional commits
- GitHub Actions CI: lint !' typecheck !' test !' build

5.3 CI/CD Pipeline (GitHub Actions)

```

# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20',
cache: 'npm' }
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - run: npm run test:coverage
      - run: npm run build
  deploy-preview:
    needs: quality
    if: github.event_name ==
'pull_request'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run build
      - uses: netlify/actions/
cli@master
        with: { args: "deploy --
dir=dist --alias=pr-
${{ github.event.number }}" }
  deploy-prod:
    needs: quality
    if: github.ref == 'refs/heads/
main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run build
      - uses: netlify/actions/
cli@master
        with: { args: "deploy --
dir=dist --prod" }

```

6. PILAR 5: ACESSIBILIDADE (WCAG 2.1 AA)

-
- Focus trap no Modal (useFocusTrap hook)
 - Skip link "Pular para conteúdo principal"
 - ARIA labels em todos os botões icon-only
 - Contraste mínimo 4.5:1 (auditar textos sobre gradients)
 - Redução de movimento: prefers-reduced-motion
 - Navegação por teclado completa (Tab, Enter, Esc)
 - Live regions para atualizações de carrinho (aria-live="polite")

- Alt texts descritivos para todas as imagens de produto

7. ROADMAP DE IMPLEMENTAÇÃO

1 - Foundation	TypeScript, ESLint/Prettier/Husky, CI, refatorar estrutura pastas, extrair componentes	2-3 semanas	CRÍTICA
2 - Core Commerce	Carrinho (Context + localStorage), Checkout	3-4 semanas	ALTA
3 - Catalog	WhatsApp melhorado, BDD (/produto/:id)	2-3 semanas	ALTA
4 - Performance/SEO	Busca, filtros, paginação/infinite scroll, estados vazios, loading skeletons	2 semanas	MÉDIA
5 - A11y/Polish	Image optimization, code-splitting, meta tags dinâmicas, JSON LD, sitemap, CMM/audit	1-2 semanas	MÉDIA
6 - Scale (Opcional)	Auditoria WCAG, focus trap, skip links, reduced motion, testes E2E, visual regression	6+ semanas	BAIXA
	Backend API (Node/Next.js), Auth, Pagamentos (MercadoPago/Stripe), Admin CMS, PWA		

8. ESTIMATIVA DE IMPACTO PÓS-MELHORIAS

Lighthouse Performance	~45	>90	+100%
Lighthouse Accessibility	~75	>95	+27%
Lighthouse SEO	~60	>95	+58%
Bundle JS (gzipped)	79 KB	<50 KB	-37%
Time to Interactive	~4.2s	<2.5s	-40%
Taxa conversão (est.)	Baixa	Média/Alta	WhatsApp otimizado + UX
Manutenibilidade	Baixa	Alta	TypeScript + testes + arquitetura
Escalabilidade	Zero	Alta	Separação dados/UI + API ready

9. CONFIGURAÇÕES DE AMBIENTE PROPOSTAS

.env.example

```
VITE_WHATSAPP_NUMBER=5521976807111
VITE_API_URL=https://api.fabulosamodas.com
VITE_GA_MEASUREMENT_ID=G-XXXXXXXXXX
VITE_SENTRY_DSN=https://xxx@sentry.io/xxx
VITE_APP_NAME="Fabulosa Modas"
```

10. CONSIDERAÇÕES FINAIS

O projeto atual demonstra excelente base visual e UX, com stack moderna e código limpo para um MVP. As melhorias propostas transformam a landing page em um e-commerce profissional, escalável e pronto para crescimento. A migração para TypeScript e a introdução de testes/CI são fundamentos inegociáveis para qualidade a longo prazo. A arquitetura baseada em Context API + hooks customizados evita over-engineering inicial (Redux/Zustand) mas permite migração futura. O roadmap em 6 fases permite entregas incrementais de valor, com a Fase 1-3 já entregando um e-commerce funcional completo.